3D Gaussian Splatting for Temporal AA, Super Resolution, and Motion-Accurate Frame Interpolation

vkGSplat research note

Started May 4, 2026

1 Executive Thesis

The idea is plausible and worth a serious research branch:

> Use 3D Gaussian Splatting not merely as a renderer, but as an explicit spatio-temporal reconstruction state for temporal anti-aliasing, temporal super-resolution, frame generation, and motion-accurate frame interpolation. The goal is a compute-first counterpart to DLSS 4.5 and later: less of a black-box image upscaler, more of a renderer whose history buffer is a persistent, splattable 3D/4D representation.

As of May 2026, DLSS 4.5 is a real product target: NVIDIA describes it as a second-generation transformer model for Super Resolution plus Dynamic Multi Frame Generation and 6X Multi Frame Generation on supported RTX 50-series GPUs [1, 2]. The research question for vkGSplat is not whether an independent project can out-train NVIDIA's network. The sharper question is whether a renderer that owns its scene/path state can avoid some of the ambiguity that image-space temporal reconstruction must learn around disocclusion, subpixel geometry, specular motion, and frame interpolation.

Current subjective likelihoods:

- Useful vkGSplat temporal reconstruction experiment: **high**, about 0.8.
- 3DGS-assisted TAA beating current CPU reprojection on static scenes: **medium-high**, about 0.65.
- 3DGS-assisted 2x temporal super-resolution prototype: **medium**, about 0.55.
- Motion-accurate 3DGS frame interpolation for rendered/captured clips: **medium-high**, about 0.65.
- 3DGS-based game-style frame generation competitive with DLSS-style MFG: **low-medium**, about 0.3.
- A full "DLSS 4.5 counterpart" without a learned image model: **low**, about 0.2.

2 Why 3DGS Might Matter for TAA/SR

Modern temporal upscalers operate on a low-resolution, jittered current frame plus engine-provided auxiliaries: depth, motion vectors, exposure, reactive masks, and history. DLSS 2 made this temporal-super-sampling pattern neural; DLSS 4 moved Super Resolution, DLAA, and Ray Reconstruction to transformer models; DLSS 4.5 upgraded the transformer model and extended frame generation [3, 1]. XeSS is explicitly described by Intel as AI-based temporal super sampling and anti-aliasing using jittered color, motion vectors, and depth [7]. AMD FSR also evolved from spatial upscaling to temporal upscaling and frame generation, and the newer FSR "Redstone" branding includes upscaling, frame generation, ray regeneration, and radiance caching [6].

The common limitation is that most of this reconstruction state is image-space or screen-derived. That is practical and general, but it means the algorithm must infer whether a sample is stable history, newly disoccluded content, a transparent particle, specular reflection, UI, or thin geometry. A 3DGS-based pipeline can carry explicit object/primitive/Gaussian identity through time. This gives the reconstruction system a stable world-space address for history.

3 Relevant Research Landscape

3.1 Anti-Aliased 3DGS

The core 3DGS renderer is not alias-free. At different resolutions or camera distances, point-sampled Gaussian evaluation can produce blur, jaggies, popping, and sampling-rate sensitivity. This is directly relevant to temporal SR: before building a DLSS-like system, the splat renderer itself must understand pixel footprint.

Multi-Scale 3D Gaussian Splatting keeps Gaussians at multiple scales so distant or low-resolution rendering can use fewer larger primitives, improving both quality and speed [9]. Mip-Splatting is the earlier must-cite alias-free baseline: it identifies sampling-rate changes, focal length, and camera distance as sources of 3DGS artifacts, then combines a 3D smoothing filter with a 2D Mip/box-filter approximation [8]. Analytic-Splatting analytically approximates Gaussian integrals over pixel windows instead of evaluating at isolated pixel centers [10]. AAA-Gaussians argues for fuller 3D evaluation through the pipeline, with adaptive 3D smoothing and stable view-space bounds to reduce aliasing, projection artifacts, and popping [11]. LOD-GS makes filtering strength sampling-rate-sensitive [12]. AA-Splat targets feed-forward 3DGS and uses opacity-balanced band-limiting for arbitrary-resolution rendering [13].

These works are mostly spatial or per-frame. The opening for vkGSplat is to combine their footprint-aware filtering with temporal history.

3.2 3DGS Super-Resolution

SplatSuRe studies selective super-resolution for multi-view consistent 3DGS, motivated by the fact that independently super-resolving input views can introduce multi-view inconsistency and blurry 3D renders [14]. 2D Gaussian Splatting has also been adapted to arbitrary-scale image super-resolution with feed-forward Gaussian prediction and scale-aware rasterization [15]. These works support the idea that Gaussian representations can be used as reconstruction carriers, not only as final scene geometry.

3.3 4D and Spacetime Gaussian Splatting

Frame generation needs a motion model. 4D Gaussian Splatting and related spacetime Gaussian methods show that Gaussians can carry time and deformation semantics. 4D-GS uses a deformation field and HexPlane-style structure for real-time dynamic scenes [16]. 4D-Rotor Gaussian Splatting uses anisotropic XYZT Gaussians and temporal slicing, reporting real-time rendering rates on RTX hardware [17]. Spacetime Gaussian Feature Splatting adds temporal opacity and parametric motion/rotation, with splatted neural features for dynamic view synthesis [18].

These papers are not DLSS replacements, but they provide a vocabulary for Gaussian frame interpolation: interpolate or slice a 4D/temporal Gaussian state, then splat the intermediate image.

3.4 Motion-Accurate Frame Interpolation

This deserves its own category. A normal frame-generation system can be perceptually smooth while being motion-inaccurate: it may hallucinate an intermediate image that looks plausible but does not preserve real object trajectories, depth ordering, or visibility events. For vkGSplat, the stronger target is *motion-accurate interpolation*: synthesize an intermediate frame by advancing explicit Gaussian state through a physically or geometrically meaningful motion model.

RetimeGS is the closest current reference. It frames arbitrary-timestamp rendering as continuous-time 4DGS reconstruction and identifies poor inter-frame rendering in prior 4DGS methods as temporal aliasing [19]. Its project page emphasizes relatively large inter-frame motion, coarse flow-guided primitive correspondences, triple-rendering supervision, spline trajectories, and dynamic stretching as tools for ghost-free retiming. Its stated limitation is also instructive: extreme rapid motion can still break optical-flow correspondences.

Other motion-aware Gaussian papers suggest useful mechanisms. Gaussian-Flow models dynamic Gaussian particles with time/frequency-domain deformation [20]. GaussianFlow connects Gaussian dynamics and pixel velocities for 4D content creation [21]. MotionGS uses explicit motion guidance for deformable 3DGS [22]. KF-GS uses Kalman-filter-style prediction and correction to improve dynamic motion modeling [23]. These works point toward a frame interpolation system whose core data is not only optical flow between two images, but per-Gaussian position, velocity, covariance, opacity, and visibility confidence.

The broader video-frame-interpolation literature is also useful. AceVFI classifies VFI methods into center-time frame interpolation and arbitrary-time frame interpolation, and identifies large motion, occlusion, lighting change, and non-linear motion as central challenges [24]. Those are the same failure modes a 3DGS interpolation system must measure, but 3DGS gives us a stronger state variable than two images and a flow field.

3.5 DLSS, FSR, and XeSS as the Bar

DLSS 3 frame generation used current/prior frames, optical flow, and engine data such as motion vectors and depth [5]. DLSS 4 introduced Multi Frame Generation and transformer models for Super Resolution, DLAA, and Ray Reconstruction [3, 4]. DLSS 4.5 then introduced a second-generation transformer Super Resolution model and Dynamic Multi Frame Generation / 6X MFG [1].

This sets a very high bar. A 3DGS counterpart should not begin by trying to replicate the whole neural stack. It should attack the part DLSS cannot explicitly own: persistent scene-space reconstruction state.

4 Candidate Architectures

4.1 A: Gaussian History Buffer

Replace or augment the image-space history buffer with persistent Gaussian history records:

$$H_j = (\mu_j, \Sigma_j, q_j, c_j, f_j, v_j, m_j, \tau_j, id_j),$$

where μ_j and Σ_j are world-space mean/covariance, q_j is orientation or tangent frame, c_j is accumulated color/radiance, f_j is a feature vector, v_j is variance, m_j is sample mass, τ_j is last update time, and id_j is primitive/Gaussian identity.

For each jittered low-resolution frame:

1. unproject visible pixels using depth and camera matrices,
2. attach primitive ID / material ID / motion vector / confidence,
3. update nearby Gaussian history entries,
4. splat the history entries into target-resolution pixels,
5. blend with current samples using depth, primitive ID, velocity, and variance gates.

This is the cleanest first vkGSplat experiment because the repo already has reprojection contracts in `include/vkgsplat/reprojection.h` and `src/core/reprojection.cpp`. The current code reprojects previous color in screen space and rejects by depth/primitive mismatch. A Gaussian history buffer keeps the same rejection semantics but moves the address space from pixels to stable 3D samples.

4.2 B: Gaussian Temporal Super-Resolution

Render low-resolution jittered frames. Each frame contributes subpixel samples to a target-resolution Gaussian reconstruction layer. Instead of resolving history by clamped pixel neighborhoods, resolve by projected Gaussian footprint:

$$C(p) = \frac{\sum_j w_j(p,t)\, c_j}{\sum_j w_j(p,t) + \epsilon}.$$

Here w_j should include:

$$w_j = w_{\text{footprint}} w_{\text{depth}} w_{\text{normal}} w_{\text{primitive}} w_{\text{velocity}} w_{\text{variance}}.$$

The important difference from TAAU is that the accumulated subpixel evidence is not tied to last frame's integer pixels. It is tied to world-space Gaussian support and can be re-sampled at the output resolution each frame.

4.3 C: Gaussian Residual Layer for DLSS-Style Reconstruction

A hybrid architecture may be strongest:

$$\begin{aligned}&\text{low-res color/depth/velocity} + \text{image history} + \text{Gaussian history features}\\ &\rightarrow \text{small reconstruction model.}\end{aligned}$$

The model does not need to infer everything from pixels. Gaussian history can provide stable features:

- accumulated radiance,
- variance/confidence,
- primitive identity consistency,
- stable world position,
- visibility age,
- disocclusion likelihood,
- high-resolution subpixel coverage.

This is the most plausible route to a DLSS-like result, but it is a later-stage branch because it needs training data and a small inference runtime.

4.4 D: Motion-Accurate Gaussian Frame Interpolation

For interpolation between rendered frames t_0 and t_1, construct an intermediate Gaussian state at t_α. The simplest model is linear:

$$\mu_j(t_\alpha) = (1-\alpha)\mu_j(t_0) + \alpha\mu_j(t_1)$$

with analogous interpolation for rotation, scale, opacity, color/features, and visibility confidence. Render by splatting the interpolated state.

But a motion-accurate system should move beyond linear interpolation. A better per-Gaussian state is:

$$M_j(t) = (\mu_j(t), \dot{\mu}_j(t), \Sigma_j(t), R_j(t), \alpha_j(t), c_j(t), \nu_j(t)),$$

where $\dot{\mu}_j$ is velocity and ν_j is visibility confidence. The trajectory can be constant-velocity, Kalman predicted/corrected, spline-based as in RetimeGS, or driven by engine object transforms. The renderer should also produce a forward and backward motion-consistency residual:

$$e_j = \left\| \Pi_{t_1}(\mu_j(t_1)) - (\Pi_{t_0}(\mu_j(t_0)) + u_{0\to 1}) \right\|,$$

where Π_t projects to the camera at time t and $u_{0\to 1}$ is engine or optical flow. High residuals mark Gaussians that should not be trusted for interpolation.

This is the direct 3DGS counterpart to video frame interpolation. It should be easier for rigid camera motion and object transforms than for particles, transparency, specular effects, animated shading, or UI. A robust system likely needs two paths:

- geometry-aware Gaussian interpolation for stable scene content,
- image-space optical-flow or neural residual generation for everything the Gaussian state cannot explain.

4.5 E: Minimal vkGSplat Data Contracts

The implementation should begin as a reference contract, not a new renderer. A useful first pair of structs:

```
struct GaussianHistorySample {
    float3 position;
    float3 normal;
    float3 color;
    float3 radiance_feature;
    float2 current_to_previous_px;
    float depth;
    float variance;
    uint32_t primitive_id;
    uint32_t material_id;
    uint32_t frame_index;
};

struct GaussianMotionState {
    float3 position_t0;
    float3 position_t1;
    float3 velocity_t0;
    float3 velocity_t1;
    float4 rotation_t0;
    float4 rotation_t1;
    float3 scale_t0;
    float3 scale_t1;
    float opacity_t0;
    float opacity_t1;
    float visibility_confidence;
    float flow_residual_px;
    uint32_t primitive_id;
    uint32_t motion_group_id;
};
```

`GaussianHistorySample` is the TAA/SR side: it turns current reprojection outputs into persistent world-space evidence. `GaussianMotionState` is the interpolation side: it stores enough state to synthesize a midpoint and to reject motion that disagrees with engine or optical flow.

5 Disocclusion and Ghosting

The central failure mode of all temporal reconstruction is stale history. The 3DGS advantage is that stale history can be tested in more ways than screen clamping:

- depth agreement after projecting the Gaussian into the current frame,
- primitive/Gaussian ID agreement,
- normal and tangent-frame agreement,

- opacity/transmittance ordering agreement,
- current visibility confidence,
- age since last observation,
- sample variance and neighborhood disagreement.

The danger is that 3DGS can make ghosting worse if old Gaussians remain visible after an object moves or a surface becomes occluded. The cache must support decay, visibility invalidation, and possibly per-camera frustum evidence.

6 vkGSplat Experiment Plan

6.1 Stage 0: Baseline TAAU Contract

Extend the current reprojection module into a minimal temporal AA/upscaling contract:

- add Halton or R2 jitter to seed frames,
- render low resolution and output high resolution,
- reproject history using camera motion map,
- add neighborhood clamp or variance clamp,
- measure against high-resolution supersampled reference.

This is the control. Without it, a Gaussian method has no honest baseline.

6.2 Stage 1: Static Gaussian History Cache

Implement CPU-only:

- visible pixel $\rightarrow$ world sample,
- world samples update Gaussian cache cells,
- cache splats to target resolution,
- compare against screen-space TAA and current reprojection.

Start with static scenes and camera-only motion. Use primitive-ID gates from the existing vkGSplat reprojection path.

6.3 Stage 2: 2x Temporal Super-Resolution

Render at half resolution with jitter and reconstruct full resolution. Compare:

- nearest/bilinear upscale,
- screen-space TAAU,
- Gaussian history resolve,
- Gaussian history plus small spatial filter.

Metrics should include PSNR/SSIM/LPIPS/FLIP where available, plus temporal metrics: shimmer under slow camera pans, ghosting after disocclusion, and edge stability on thin geometry.

6.4 Stage 3: Dynamic Gaussian Motion

Add object transforms and per-Gaussian velocity:

- update Gaussian centers/covariances through object motion,
- invalidate history on topology/primitive mismatch,
- compare against engine motion-vector reprojection,
- test on moving rigid objects before deformation.

6.5 Stage 4: Motion-Accurate Gaussian Frame Interpolation

Generate one intermediate frame between two rendered frames:

- interpolate/slice Gaussian state at a chosen $\alpha \in (0, 1)$,
- start with engine-known camera/object transforms and constant velocity,
- add spline/Kalman trajectories for non-linear motion,
- validate projected Gaussian motion against engine or optical flow,
- splat intermediate view,
- generate a disocclusion mask from forward/backward Gaussian visibility,
- composite reactive image-space residuals for particles, transparency, specular effects, and UI,
- compare against optical-flow interpolation and native middle frame.

This should stay separate from the TAAU work until Stage 2 is healthy. The first proof point is not "more frames" but whether the generated midpoint lands moving edges, occlusion boundaries, and object silhouettes in the same place as a true rendered middle frame.

6.6 Stage 5: Learned Resolver

Only after the analytic resolver shows signal:

- train a compact model on low-res inputs plus Gaussian history features,
- use high-res/high-spp references from vkGSplat or Wicked Engine,
- keep inference compatible with CUDA/Metal/Triton-style kernels,
- compare with non-Gaussian neural TAAU using the same training data.

7 Key Risks

- The method may become a heavier version of TAA without enough quality gain.
- Gaussian history can smear across visibility boundaries unless gating is strict.
- Specular and transparent effects may remain image-space problems.
- Dynamic objects need robust lifetime and identity management.
- A full DLSS 4.5-class model needs large training data and careful latency engineering.
- 3DGS visibility ordering can fight temporal stability if splats pop, split, merge, or change opacity under optimization.

8 Current Recommendation

Do pursue this, but sequence it conservatively:

> First build a strong screen-space TAAU reference. Then add a CPU Gaussian history cache for static camera-motion scenes. If that beats TAAU on shimmer, thin geometry, and disocclusion without huge cost, move it to CUDA/Metal and only then explore learned resolving and frame generation.

The best vkGSplat framing is not "we clone DLSS." It is:

> vkGSplat explores explicit 3D/4D reconstruction state as a compute-first alternative to purely image-space temporal neural rendering.

9 Overnight Research TODO

- Read full papers for Multi-Scale 3DGS, Analytic-Splatting, AAA-Gaussians, LOD-GS, AA-Splat, SplatSuRe, RetimeGS, and 4DGS variants.
- Search for direct "Gaussian temporal super-resolution" or "Gaussian TAA" papers that may not surface under 3DGS keywords.
- Compare DLSS 4.5, AMD FSR Redstone, and XeSS 3-style multi-frame generation claims using primary sources where possible.
- Extract RetimeGS details for continuous-time opacity, spline trajectories, flow-guided correspondences, triple rendering, and dynamic stretching.
- Sketch a `GaussianHistoryBuffer` C++ contract compatible with current vkGSplat reprojection and denoise frame structs.
- Sketch a `GaussianMotionState` contract for midpoint synthesis: position, velocity, covariance, opacity, visibility, and residual flow.
- Identify the minimum benchmark clip needed to expose shimmer, disocclusion, thin geometry, and moving-object ghosting.

References

[1] NVIDIA. DLSS 4.5 Dynamic Multi Frame Generation and Multi Frame Generation 6X Available Now. March 31, 2026. https://www.nvidia.com/en-us/geforce/news/dlss-4-5-dynamic-multi-frame-generation-6x-mode-released/

[2] NVIDIA Developer. NVIDIA DLSS. Accessed May 4, 2026. https://developer.nvidia.com/rtx/dlss

[3] NVIDIA. NVIDIA DLSS 4 Introduces Multi Frame Generation and Enhancements for All DLSS Technologies. January 6, 2025. https://www.nvidia.com/en-sg/geforce/news/dlss4-multi-frame-generation-ai-innovations/

[4] NVIDIA Research. DLSS 4: Transforming Real-Time Graphics with AI. Technical report, 2025. https://research.nvidia.com/labs/adlr/DLSS4/

[5] NVIDIA. Introducing NVIDIA DLSS 3. September 20, 2022. https://www.nvidia.com/en-us/geforce/news/dlss3-ai-powered-neural-graphics-innovations/

[6] AMD. AMD FSR Technologies. Accessed May 4, 2026. https://www.amd.com/en/products/graphics/technologies/fidelityfx/super-resolution.html

[7] Intel. Intel XeSS Super Resolution Developer Guide 2.0. Accessed May 4, 2026. https://www.intel.com/content/www/us/en/developer/articles/technical/xess-sr-developer-guide.html

[8] Zehao Yu, Anpei Chen, Binbin Huang, Torsten Sattler, and Andreas Geiger. Mip-Splatting: Alias-Free 3D Gaussian Splatting. CVPR, 2024. https://openaccess.thecvf.com/content/CVPR2024/html/Yu_Mip-Splatting_Alias-free_3D_Gaussian_Splatting_CVPR_2024_paper.html

[9] Zhiwen Yan, Weng Fei Low, Yu Chen, and Gim Hee Lee. Multi-Scale 3D Gaussian Splatting for Anti-Aliased Rendering. CVPR, 2024. https://cvpr.thecvf.com/virtual/2024/poster/30347

[10] Zhihao Liang, Qi Zhang, Wenbo Hu, Ying Feng, Lei Zhu, and Kui Jia. Analytic-Splatting: Anti-Aliased 3D Gaussian Splatting via Analytic Integration. arXiv:2403.11056, 2024. https://arxiv.org/abs/2403.11056

[11] Michael Steiner, Thomas Koehler, Lukas Radl, Felix Windisch, Dieter Schmalstieg, and Markus Steinberger. AAA-Gaussians: Anti-Aliased and Artifact-Free 3D Gaussian Rendering. arXiv preprint, 2025. https://tugraz.elsevierpure.com/en/publications/aaa-gaussians-anti-aliased-and-artifact-free-3d-gaussian-renderin/

[12] Zhenya Yang, Bingchen Gong, and Kai Chen. LOD-GS: Level-of-Detail-Sensitive 3D Gaussian Splatting for Detail Conserved Anti-Aliasing. arXiv:2507.00554, 2025. https://papers.cool/arxiv/2507.00554

[13] Taewoo Suh, Sungpyo Kim, Jongmin Park, and Munchurl Kim. AA-Splat: Anti-Aliased Feed-forward Gaussian Splatting. arXiv:2603.29394, 2026. https://papers.cool/arxiv/2603.29394

[14] Pranav Asthana, Alex Hanson, Allen Tu, Tom Goldstein, Matthias Zwicker, and Amitabh Varshney. SplatSuRe: Selective Super-Resolution for Multi-view Consistent 3D Gaussian Splatting. CVPR, 2026. https://splatsure.github.io/

[15] Du Chen, Liyi Chen, Zhengqiang Zhang, and Lei Zhang. Generalized and Efficient 2D Gaussian Splatting for Arbitrary-scale Super-Resolution. arXiv:2501.06838, 2025. https://huggingface.co/papers/2501.06838

[16] Guanjun Wu, Taoran Yi, Jiemin Fang, Lingxi Xie, Xiaopeng Zhang, Wei Wei, Wenyu Liu, Qi Tian, and Xinggang Wang. 4D Gaussian Splatting for Real-Time Dynamic Scene Rendering. CVPR, 2024. https://guanjunwu.github.io/4dgs/

[17] Yuanxing Duan, Fangyin Wei, Qiyu Dai, Yuhang He, Wenzheng Chen, and Baoquan Chen. 4D-Rotor Gaussian Splatting: Towards Efficient Novel View Synthesis for Dynamic Scenes. 2024. https://research.nvidia.com/publication/2024-02_4d-rotor-gaussian-splatting-towards-efficient-novel-view-synthesis-dynamic

[18] Zhan Li, Zhang Chen, Zhong Li, and Yi Xu. Spacetime Gaussian Feature Splatting for Real-Time Dynamic View Synthesis. arXiv:2312.16812, 2023. https://arxiv.org/abs/2312.16812

[19] Xuezhen Wang, Li Ma, Yulin Shen, Zeyu Wang, and Pedro V. Sander. RetimeGS: Continuous-Time Reconstruction of 4D Gaussian Splatting. CVPR, 2026. https://william-wang2.github.io/RetimeGS/

[20] Youtian Lin, Zuozhuo Dai, Siyu Zhu, and Yao Yao. Gaussian-Flow: 4D Reconstruction with Dynamic 3D Gaussian Particle. arXiv:2312.03431, 2023. https://huggingface.co/papers/2312.03431

[21] Quankai Gao, Qiangeng Xu, Zhe Cao, Ben Mildenhall, Wenchao Ma, Le Chen, Dahang Tang, and Ulrich Neumann. GaussianFlow: Splatting Gaussian Dynamics for 4D Content Creation. arXiv:2403.12365, 2024. https://huggingface.co/papers/2403.12365

[22] Ruijie Zhu, Yanzhe Liang, Hanzhi Chang, Jiacheng Deng, Jiahao Lu, Wenfei Yang, Tianzhu Zhang, and Yongdong Zhang. MotionGS: Exploring Explicit Motion Guidance for Deformable 3D Gaussian Splatting. arXiv:2410.07707, 2024. https://huggingface.co/papers/2410.07707

[23] Qingyuan Tang, Yufei Yin, Yanming Zhu, Zhou Yu, Zhenzhong Kuang, Jiajun Ding, and Jifa He. KF-GS: Kalman Filter-Guided Gaussian Splatting for Real-Time High-Quality Dynamic Scene Reconstruction. Journal of Visual Communication and Image Representation, in press, 2026. https://www.sciencedirect.com/science/article/pii/S1047320326001100

[24] Dahyeon Kye, Changhyun Roh, Sukhun Ko, Chanho Eom, and Jihyong Oh. AceVFI: A Comprehensive Survey of Advances in Video Frame Interpolation. IEEE Transactions on Circuits and Systems for Video Technology, 2026. https://www.researchgate.net/publication/401799473_AceVFI_A_Comprehensive_Survey_of_Advances_in_Video_Frame_Interpolation